AI DEVELOPMENT FRAMEWORK · v0.2

2026-10-17 · 6f4bfe1

자기 개선 개발 프레임워크

사람이 통제권을 지킨 채 AI의 개발 노동을 안전하게 쓰는 방법 — erpsarang/self-improvement-mvp



세 부분으로 읽는다

01

배경, 목표, 의의

왜 만들었고, 무엇을 지향하며, 무엇이 다른가

03-09

02

작동 방식 A–Z

요구 Issue 하나가 검증된 변경이 되기까지의 모든 단계

10-24

03

새 App 개발 가이드

빈 저장소에서 첫 Merge까지, 0단계부터 12단계

25-34

01

PART 01 · 슬라이드 03-09

배경, 목표, 의의

AI가 만든 변경을 어떻게 믿을 것인가에서 출발했다

한 달 동안 되돌아온 세 질문

Q1

AI가 만든 변경을 어떻게 믿을 것인가

지금의 답

AI 결과는 늘 candidate다. 검증 레일을 지나야 변경이 된다.

Q2

사람의 개입을 어디까지 줄일 수 있는가

지금의 답

판단 없는 클릭은 없애고, 판단이 있는 자리는 남긴다.

Q3

프레임워크 다듬기는 언제 집착이 되는가

지금의 답

관측된 실패로 설명할 수 없는 변경은 만들지 않는다.

출처: CHARTER.md 전문 · 2026-09-06부터 약 한 달

AI가 만들었다 ≠ trusted

AI 결과는 항상 **candidate**다. 정해진 검증 레일을 지나야만 실제 소프트웨어 변경이 된다.

목표는 세 문장이다

1

신뢰할 품질, 최소의 사람 개입

품질을 지키면서 사람의 손을 최소로 쓰는 App 개발 프레임워크를 만든다.

2

App이 다음 과제를 스스로 찾는다

사용자 관점에서 과제를 찾고, 검증된 사이클로 실현하고, 결과에서 다시 배운다. 이것이 이름의 뜻이다.

3

개입은 줄이고 authority는 유지한다

개입 지점을 없애려면, 그 판단을 대신할 증거와 검증이 먼저 있어야 한다.

> **프레임워크는 수단이고 App은 목적이다.**
>
> 가치는 완성도가 아니라, App이 사용자에게 더 나아지는 속도와 신뢰성으로 잰다.

출처: CHARTER.md 제1장 목적

충돌하면 위가 이긴다

1	**증명 가능한 안전**	exact SHA · provenance · fail-closed · bounded 반복 · Human-only Merge
2	**App 사용자의 실질적 가치**	변경은 App을 쓰는 사람에게 주는 것으로 정당화한다
3	**사람 개입의 최소화**	새 판단이 없는 클릭은 없앤다
4	**지속 가능성(비용·시간)**	같은 입력의 성공 호출을 다시 부르지 않는다
5	**내부 정합성과 우아함**	가장 낮다. 매몰되지 않는다

출처: CHARTER.md 제2장 · 순위는 충돌할 때 무엇을 양보하는가의 순서다

증명할 수 있는 자동화

A

근거와 권한이 증명된다

AI가 무엇을 근거로 했고 어디까지 권한이 있었는지 SHA와 artifact로 남는다.

B

사람에게는 판단만 남는다

요구, PLAN 승인, Merge, 후보 채택. 나머지는 프레임워크가 한다.

C

STOP이 정상 상태다

한도를 넘거나 identity가 불명확하면 멈추고 사람에게 넘긴다.

D

비개발자도 요구를 넣는다

“무엇이 불편하고 어떻게 바뀌면 좋은가”만 쓰면 된다.

실제 App에서 증명

3

개 App에서 요구부터 Merge·LEARN까지

878

Framework 커밋

650

App 커밋

수치: CHARTER.md 부록 A (2026-10-02 기준) · App: sales-order-exception-analyzer, meeting-action-items, transportation_guide

우리가 만들지 않는 것

× 01

~~완전자율 개발 AI~~

× 02

~~AI의 자기 승인~~

× 03

~~무제한 FIX 루프~~

× 04

~~Auto Merge~~

× 05

~~무한 Self-Improvement~~

× 06

~~blocker 재귀~~

대신 Self-Improvement는 자율적인 자기 수정이 아니라 **증거에 근거한 개선 제안**이다.

출처: docs/architecture.md 3장, 7장

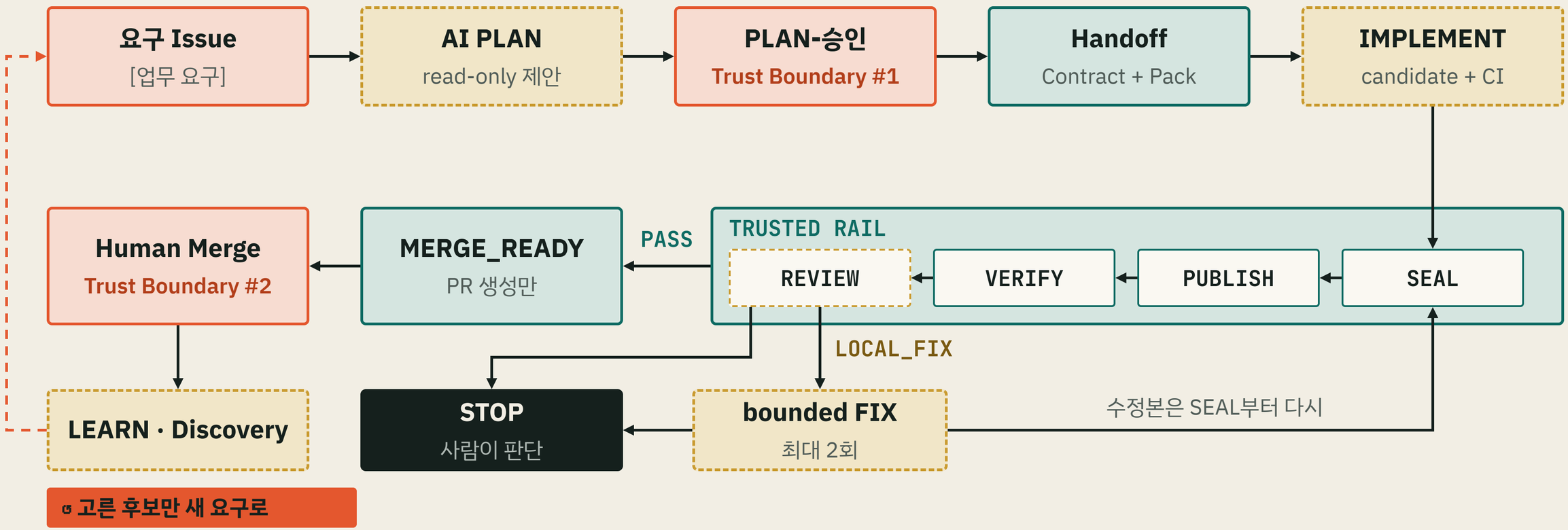
02

PART 02 · 슬라이드 10-24

작동 방식 A–Z

요구 Issue 하나가 검증된 변경이 되기까지

요구에서 Merge까지, 한 사이클



■ 사람의 판단 ■ AI 제안 ■ Trusted 검증 · 출처: docs/architecture.md 1장 · README v0.2 흐름

저장소 세 개가 역할을 나눈다



App workflow는 AI를 직접 부르지 않는다. 결과는 trusted 단계가 다시 검증한다.

출처: docs/architecture.md 4장 · docs/usage.md 2장 · 예: erpsarang/transportation_guide

누가 무엇을 하고, 하지 않는가

주체	하는 일	하지 않는 일
사람	요구 작성 · PLAN-승인 · 최종 Merge · 후보 채택/기각 · STOP 뒤 판단	판단 없는 클릭 반복
read-only AI Planner · Learner · Evaluator	bounded Context를 읽고 제안	저장소 수정, 자기 제안 승인
untrusted Worker IMPLEMENT · FIX	승인된 scope 안에서 candidate 생성	write credential, scope 밖 변경, 직접 publish
Semantic Reviewer	exact SHA를 읽고 판단을 제안	판단 확정, push, merge
Trusted control-plane	결정적 검증 · provenance 기록 · routing	AI 판단, 사람 authority 대체

출처: CHARTER.md 제7장 · 프레임워크 안의 AI는 저장소 문서를 지시로 읽지 않는다

요구가 들어오면 PLAN이 나온다

A 요구 접수

- 제목 [업무 요구] Issue를 열면 PLAN이 자동 시작
- 저장소 구성원이 연 Issue만 → 외부인에 AI 비용 없음
- Discovery 후보도 같은 PLAN으로, 출처는 source 기록
- Requirement Lint가 AI 없이 R1–R4 조언 댓글

B Read-only AI PLAN

`12 files · 80KB` `allowedPaths ≤ 8`

- sonnet 기본, “복잡한 요구” 체크 시 opus
- 검증 명령은 npm test · npm run build만
- trusted 검증이 근거·경로 대조, 영향 테스트 보강

C 재시작 · 재시도 · 복구 — 모두 상한이 있다

4 본문을 고치면 재PLAN
Issue당 다른 본문 4개

2 EXECUTION_FAILED만 재시도
(Issue, digest)당 2회

2 승인 뒤 base가 움직이면 복구
승인당 2회, 재승인 필요

출처: docs/usage.md 1-1장·4장 · plan.yml · plan-retry.ts · plan-recovery.ts

승인된 PLAN만 구현 권한이 된다

D · 사람

PLAN-승인

사람이 정확히 이 한 줄을 댓글로 남긴다.

digest · run · artifact · target SHA · 댓글 ID를 다시 검증

Trust Boundary #1

→

E · Trusted

Handoff

scope를 재해석 없이 ImplementContract로 바꾼다.

원문 384KB 보관, Worker prompt엔 96KB까지

→

F · AI

IMPLEMENT

credential 없이 edit JSON만 낸다.

경로·anchor 검사 후 CI. 실패하면 repair 최대 2회

→

G · Trusted

PLAN Bridge

CI를 통과한 candidate를 Rail 입력으로.

PLAN provenance를 그대로 보존

fail-closed · 승인 뒤 본문이나 target SHA가 바뀌면 그 승인은 거부된다. 조용히 대체하지 않는다.

출처: docs/usage.md 5–6장 · plan-authorize.yml · plan-implement-worker.yml

Trusted Rail의 네 단계

SEAL

candidate를 실행하지 않고
identity·base·bytes digest를
검증해 봉인한다

→

PUBLISH

sealed patch만 ai-publish/
브랜치에 force 없이 올린다.
main에는 push하지 않는다

→

VERIFY

1 prepare · exact SHA 결정
2 candidate · 격리, secret 없음
3 finalize · 새 runner에서

→

REVIEW

1 prepare · digest 재계산
2 agent · opus 정적 검토
3 finalize · 일관성 검사

read **write** **read** **read**

PASS	LOCAL_FIX	STRUCTURAL
BLOCKER 0개 → MERGE_READY	모두 LOCAL → FIXING, 최대 2회	STRUCTURAL BLOCKER → STOPPED

출처: docs/trust-model.md · trusted-rail.yml · semantic-review.yml · 권한은 job 단위로 준다

고치거나 멈추고, 사람 앞에 놓는다

I · ORCHESTRATOR · FIX

판단은 이미 끝났다. 결정적으로 보낸다

LOCAL_FIX면 allowedPaths 원문과 LOCAL BLOCKER만 sonnet에 1회. 수정본은 SEAL부터 Rail 전체를 다시 지난다.

2 번까지. 세 번째는 STOPPED

J · MERGE_READY PR

권한이 merge를 구조적으로 막는다

- ✓ `pull-requests: write`
- ✓ `contents: read`
- × `merge · push · auto`

PASS일 때만. reviewed SHA = branch HEAD = PR head를 강제한다.

K · HUMAN MERGE

확인하고, 사람이 Merge한다

- ☐ PR HEAD = reviewed SHA
- ☐ 변경 파일이 승인 범위 안
- ☐ CI · provenance 정상
- ☐ 운영·보안 영향 없음

Trust Boundary #2

출처: docs/trust-model.md Orchestrator·Merge · docs/usage.md 2-1장, 8–9장

Merge 뒤에는 배우고 제안한다

L · LEARN

사이클이 어떻게 흘렀나

01 실제 Merge만 감지한다. open PR은 완료가 아니다

02 Completed Cycle Record와 bounded Input Pack

03 Learner(sonnet)가 Pack만 읽고 evidence ID에 근거한다

04 proposal-only 후보. 자동 Issue·구현은 없다

M · PRODUCT DISCOVERY

App이 사용자에게 충분한가

131KB
Snapshot

→

3→1
opus 비교

→

≤ 1
Issue

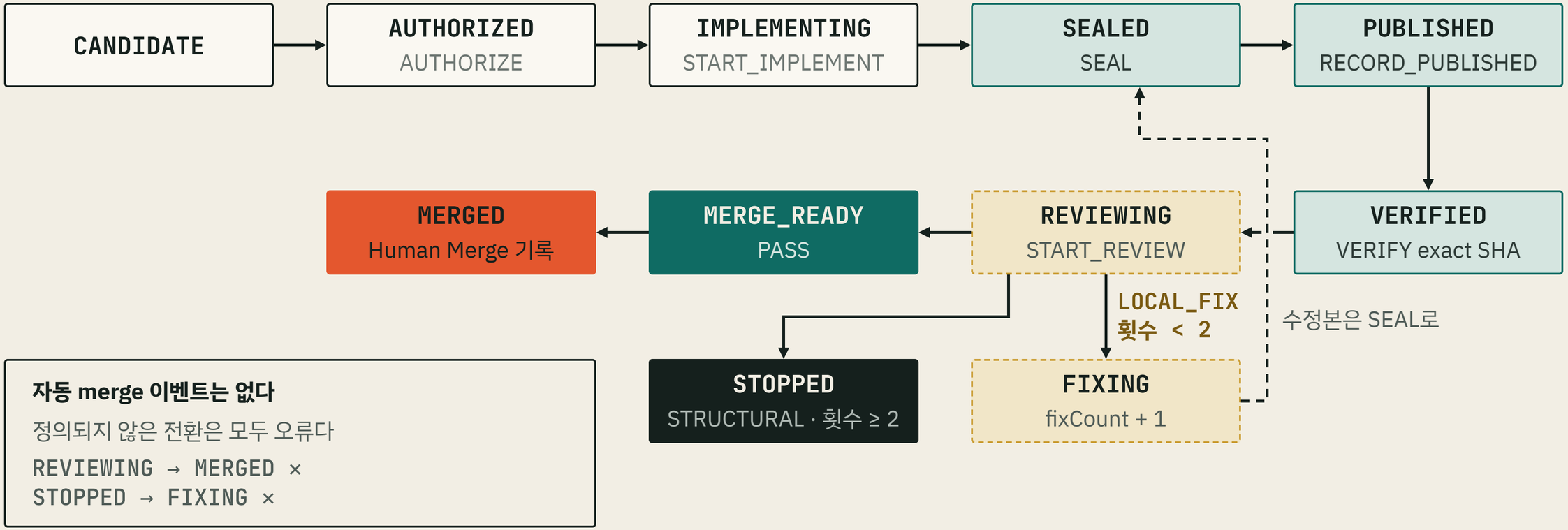
제품 source 전체·README·최근 이력을 본다. Framework 파일은 빠진다.

자동 시작(opt-in): 모두 만족할 때

Merge 3건 · 열린 후보 없음 · 24시간 · 실행 중 run 없음

출처: docs/usage.md 10–11-1장 · not_planned와 사유로 닫은 후보는 다시 제안되지 않는다

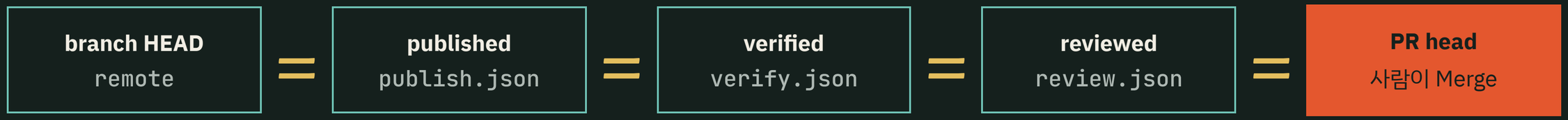
정의된 전환만 허용된다



출처: docs/state-machine.md · src/self-improvement/state.ts

모든 값이 같아야 넘어간다

exact SHA 사슬



요구 digest도 같아야 한다 · `review.requirementsDigest == authorize.requirements.digest`

01

권한 분리

contents:write는 PUBLISH job 하나, pull-requests:write는 PR job 하나에만

02

runner 분리

candidate를 실행하는 runner엔 handler·secret·credential이 없다

03

trust anchor

Actions artifact. Issue 댓글이나 현재 브랜치 텍스트는 신뢰 입력이 아니다

출처: docs/trust-model.md VERIFY·REVIEW·Merge

AI 호출은 팀 executor에서

단계	모델	호출	도구
PLAN	sonnet · opus*	요청당 1회	스위치 시 읽기
IMPLEMENT	sonnet	1회 + repair ≤ 2	스위치 시 읽기
REVIEW	opus	Rail당 1회	없음
FIX	sonnet	1회 × 최대 2번	없음
LEARN	sonnet	Merge당 1회	없음
Discovery	opus	1회, 재시도 없음	없음

* “복잡한 요구” 체크 시 opus · effort medium · 읽기 도구는 Read·Glob·Grep만, 격리된 checkout 안

ISOLATION

구독과 자격은 공유하지 않는다

Framework·App, App·App이 같은 executor나 계정을 쓰지 않는다. allowlist 밖은 호출 전 거부

TIMEOUT

기다림에도 상한

결과를 약 15분(PLAN 10분) 기다린다. FAILED marker를 보면 즉시 멈춘다

출처: docs/architecture.md 4장 · docs/usage.md 2장 · CHARTER.md 부록 A 모델 배치

판단이 적은 개입부터 줄인다

남긴다	**판단이 있는 개입**	PLAN-승인(#1) · 최종 Merge(#2) · 후보 채택·기각 · STOP 뒤 판단
조건부	**시작 trigger**	요구 Issue 작성 · Product Discovery 실행(opt-in 자동 시작)
없앤다	**운영 수작업**	bot PR CI 승인 · 멈춘 Worker 복구 · 동기화 commit · 사용량 수집

STOP 조건

멈추는 것이 정상이다

- bounded repair 소진
- Framework·control-plane 결함
- blocker가 또 blocker를 요구
- identity·provenance 불명확
- 비용·시간 budget 초과

출처: CHARTER.md 제10장 · docs/architecture.md 5장

누가 발견했느냐로 경로가 갈린다

HUMAN-DIRECTED MAINTENANCE

사람이 무엇을 바꿀지 이미 정했다

문서 현행화 · 작은 유지보수 · 관측된 Framework blocker 복구. AI 세션과 직접 branch·PR. 범위 확인, CI, Human-only Merge는 그대로

SELF-IMPROVEMENT

App이 개선 대상을 스스로 발견했다

LEARN · Product Discovery 후보 → 사람 선택 → PLAN 사이클. 제안일 뿐, 자기 승인은 없다

프레임워크를 바꾸려면 한 문장으로 설명돼야 한다

관측된 실패

Issue·run·SHA를 가리킨다

목적을 향한 전진

개입·비용 감소, 증거 강화

경계 유지

Framework와 App의 분리

완료 조건은 테스트 통과가 아니라 실제 App에서 한 사이클이 끝까지 도는 것

출처: README “Self-Improvement 적용 경계” · CHARTER.md 제5장

지금 알고 있는 한계

1 **GitHub + Node 전제**

다른 언어는 npm test·build가 그 언어를 부르게 해서만 쓴다. 예: 표준 라이브러리만 쓰는 Python App

2 **승인자 검사가 얕다**

계정 유형 User만 본다. 공개 저장소에선 외부인의 PLAN-승인도 PR까지 간다. Merge는 여전히 사람만

3 **개인 계정 소유 App만**

결과 댓글을 저장소 소유자 계정 것만 받는다. Organization 소유 App은 아직 미지원

4 **v0.3 이후 후보**

범용 GRAPH 엔진 · GitHub 외 adapter · Node 외 verifier · 장기 provenance 저장소 · 운영 UI

출처: docs/trust-model.md · docs/usage.md 2장·2-2장 · docs/roadmap.md v0.3

03

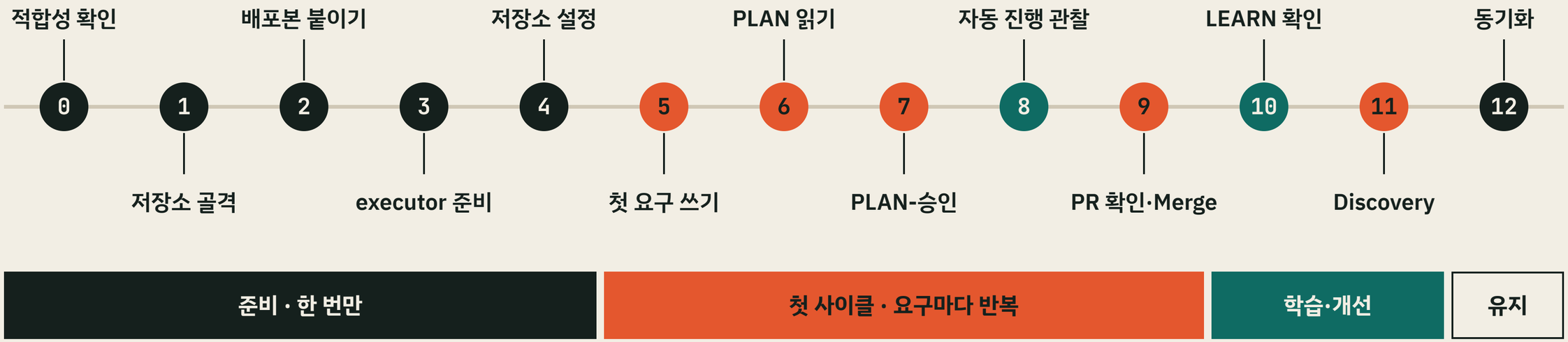
PART 03 · 슬라이드 25-34

새 App 개발 가이드

빈 저장소에서 첫 Merge까지, 0단계부터 12단계

열세 단계, 네 묶음

사람이 판단하는 단계 · 자동으로 도는 단계 · 설정·유지 작업



근거: docs/usage.md 2–11장 · transportation_guide와 meeting-action-items의 실제 bootstrap

맞는 App인지 보고 골격을 세운다

0단계 · 코드가 강제하는 조건

01 루트 package.json에 **test**와 **build**

02 package.json · tsconfig.json은 PLAN이 항상 읽는다

03 코드는 src/, 테스트는 test/ 또는 *.test.*

04 App 저장소는 **개인 계정** 소유

검증된 환경: Node 22 + TypeScript, 외부 API·자체 secret 없음. 다른 언어는 표준 라이브러리만.

1단계 · 저장소와 골격

```
my-app/
├── README.md 제품 설명서
├── package.json test · build
├── src/ 런타임 코드
└── test/ 자동 테스트
```

README가 중요한 이유

PLAN과 Discovery는 README에서 App의 목적을 읽는다. 사용자, 입력, 결과, 단계별 범위, 기술 제약을 적는다.

출처: docs/usage.md 2-2장 “App이 지켜야 할 것”

배포본을 붙인다

.github/workflows/*	Framework · 19
src/self-improvement/	Framework
policy/ · .framework-runtime/	Framework
FRAMEWORK.md	표식 · source SHA
.github/workflows/ci.yml	App
.github/ISSUE_TEMPLATE/	App
src/<app>/ · test/ · README.md	App
package.json · package-lock.json	App

1. Framework 기본 브랜치의 최신 SHA를 고른다
2. 소유 목록 **85 entry**를 sourcePath → targetPath로 복사. Framework의 package.json·lock은 .framework-runtime/으로
3. 원본과 **바이트 단위로 대조**한다
4. 목록 밖 세 파일: Issue 템플릿, ci.yml, FRAMEWORK.md
5. 루트 devDependencies에 tsx · typescript · @types/node

bootstrap은 PR로 올리고 사람이 Merge한다

Framework 소유 App 소유 · 근거: policy/framework-distribution-ownership.v1.json · transportation_guide 커밋 dcff4f1

팀 executor를 만들고 연결한다

3단계 · executor (Private)

1. subscription-ai-executor를 팀 계정에 Private 복제
2. EXECUTOR_ALLOWED_REPOSITORIES = App 저장소
3. FRAMEWORK_BRIDGE_TOKEN: App 소유 계정의 Issues RW · Actions R
4. self-hosted runner + **팀 전용** Claude 로그인
5. 선택: PLAN·IMPLEMENT 격리 읽기 도구 스위치

4단계 · App 저장소 설정

AI_EXECUTOR_REPOSITORY	필수
EXECUTOR_DISPATCH_TOKEN	필수
MERGE_READY_APP_ID · KEY	권장
TRUSTED_PUBLISH_TOKEN	조건부
AUTO_PRODUCT_DISCOVERY=on	선택
AUTO_REPLAN=off	선택

Merge-Ready App이 없으면 bot PR의 CI를 사람이 한 번 승인해야 한다 · secret 값은 어디에도 적지 않는다 · 출처: docs/usage.md 2장

첫 요구는 사용자의 말로 쓴다

Issue [업무 요구] 목적지까지 가장 빠른 수단

어떤 업무가 불편한가요?

처음 가는 곳마다 어떤 수단이 빠른지 따로 찾아야 한다.

어떻게 바뀌면 좋겠나요?

주소를 넣으면 도보·자전거·자동차가 빠른 순으로 나온다.

잘 되었다고 판단할 수 있는 예

가장 빠른 수단이 맨 위에 온다. 자동 테스트가 통과한다.

복잡한 요구인가요?

- [ ] 복잡한 요구입니다

사용자 관점으로

누가, 어떤 상황에서, 무엇이 나아지는가. 구현 방법은 안 써도 된다

한 Issue는 한 변경

허용 파일은 8개까지. 보강 테스트·lock도 이 안이다

경로는 backtick으로

아는 파일은 `src/...`로, 손대지 않을 파일은 “이 Issue 밖”이라고

짧게

제목·본문 JSON이 8,192바이트를 넘으면 LEARN이 실패. 한글은 글자당 3바이트

예시: transportation_guide README의 도메인 · 출처: docs/usage.md 1장, 1-1장, 4장 · Issue 템플릿

PLAN을 읽고 납득되면 승인한다

LINT R1 오류, R3·R4 경고가 있으면 본문을 고친다 → PLAN 자동 재시작

PLAN 댓글에서 확인할 것

- ☐ 구현 접근이 요구와 맞는가
- ☐ allowedPaths와 보강 테스트가 필요한 범위인가
- ☐ required·forbidden이 완료 조건과 금지를 담았나
- ☐ ready=true, 남은 blocking question 없음

납득되지 않으면 승인하지 말고 본문을 고친다. 진행하지 않을 요구는 사유를 남기고 닫는다.

출처: docs/usage.md 1-1장, 4–5장

Issue에 이 한 줄만

PLAN-승인

구현 authority로 가는 유일한 입구

승인 뒤 본문을 고치면 그 승인은 무효. 새 PLAN에 다시 승인한다.

자동 진행 중 이 댓글이 보이면

보이는 것	뜻	할 일
PLAN_FAILED · 경로/범위	trusted 검증이 PLAN을 거부	본문 수정, 크면 Issue 분할
PLAN 자동 재시도 (n/2)	executor 실행 실패를 다시 시도	기다린다
STOPPED · timeout	runner가 job을 제때 못 집음	runner를 살리고 그 단계 재시작
STOPPED · KIND_FAILED	executor 검증·호출·게시 실패	executor poller 로그 확인
STRUCTURAL · FIX 소진	재설계 필요, 수리 한도 소진	요구를 다시 설계하거나 분할
PLAN 자동 시작 상한	서로 다른 본문 4개를 넘김	Actions에서 PLAN 직접 실행

진행 중에는 executor runner가 켜져 있어야 한다 · 상태는 Issue의 HumanStatus 댓글과 Actions에서 본다

출처: docs/usage.md 2장 주의, 4장 PLAN 재시작·재시도

Merge, 학습, 다음 과제, 동기화

9 PR 확인 후 Merge

HEAD = reviewed SHA · 범위 안 · CI·provenance 정상 · 운영·보안 영향 없음. 사람이 직접 Merge

10 LEARN 확인

닫힌 요구 Issue 댓글에 결과가 남는다. 가치 있는 후보는 사람이 새 [업무 요구]로 옮긴다

11 Product Discovery

Merge가 쌓이면 실행(또는 opt-in). 후보는 PLAN-승인하거나, **사유를 남기고 not_planned**로 닫는다

12 Framework 동기화

결함은 App에서 고치지 않는다. Framework에서 고친 SHA를 다시 복사·대조하고 FRAMEWORK.md를 갱신

출처: docs/usage.md 9–11-1장 · CHARTER.md 제6장 7항

개입은 줄이고, authority는 사람에게

STOP은 정상 상태이고, 최종 Merge는 언제나 사람이 한다.

첫 사이클 체크리스트

- [ ] 개인 계정 App 저장소 · README · test/build 통과
- [ ] 배포본 복사·바이트 대조 + 템플릿·ci.yml·FRAMEWORK.md
- [ ] Private executor · allowlist · runner + 팀 Claude
- [ ] AI_EXECUTOR_REPOSITORY · DISPATCH_TOKEN 설정
- [ ] 첫 [업무 요구] → Lint → PLAN → PLAN-승인
- [ ] PR 확인 → Human Merge → LEARN 댓글

자세히: docs/introduction.md · CHARTER.md · docs/usage.md